

From Numerical Noise to Behavioural change ,

Mechanistic Interpretability to Batch-Dependent Computation in Language Models

Hussain Salih 2026-2027
hussainsalih.github@gmail.com

Executive Summary

Problem /

When the model evaluates the same prompt and weights only change the batch size variations that potentially leads to changing models output, because kernel implementations are batch variants. I test whether numerical instability is large enough to flip model top token prediction on simple prompt (yes/on) example, and whether batch-invariant mitigates this.

Thinking Machine company made a research *“Defeating Nondeterminism in LLM Inference”* [1] , they found the root cause of this phenomena is lack of batch invariance, LLM depends on Batch Size at inference and the reason behind it operations inside kernel not fixed related to batch size , providing Batch Invariant Ops [2]

Just a small proof-of-concept, test batch invariance using (MLX-Deterministic)[3], test whether the logits or prediction of outputs remain the same.

I’m experimenting using the **Qwen2.5-0.5B-Instruct-4bit** model with MLX, comparing single forward pass logits across batch sizes [1,2,4,8,16].

Findings /

1. Batch size alone changes the top token prediction.
2. Residual stream analysis shows divergence through depths (layers)
Comparing batch-1 and batch-16 hidden states , L2 divergence grow from ~ 0 to ~ 0.73 by the final layer
3. Batch-Invariant mode prevented the flip.

If small normal numerical batch-sizes effects can flip model behaviour with simple Yes/No question, this motivated whether deterministic is viable mitigation and if so at what cost (compute) ?

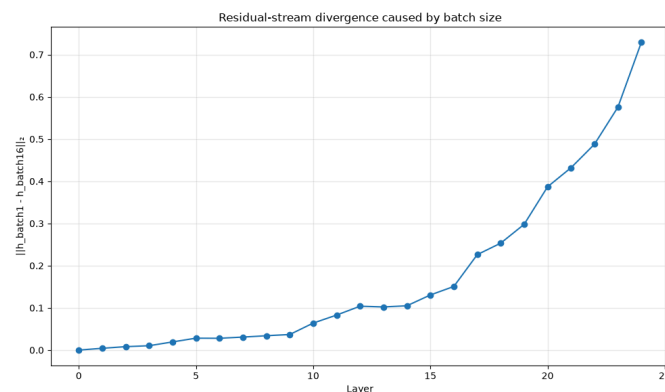
Experiments

Using *mlx-community/Qwen2.5-0.5B-Instruct-4bit* model , and with deterministic using *mlx-deterministic* following the approach of “*Defeating Nondeterminism in LLM Inference*” [1].

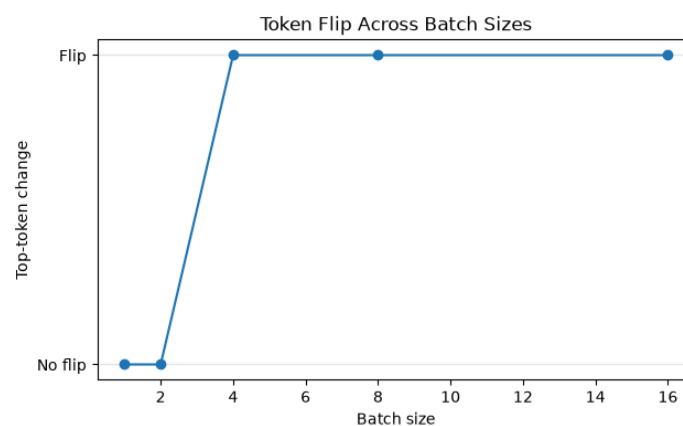
- Experiment 1 : Residual stream divergence from batch sizes

With single prompt and single forward-pass, for the 24 layers , comparing batch-1 and batch-16 the divergence (L2 norm) start from 0 at the embedding layer then grows roughly to ≈ 0.73 by the final layer, which approximately $\sim 170x$ increase from numeric perturbation.

That shows unstable computation propagating through layers.

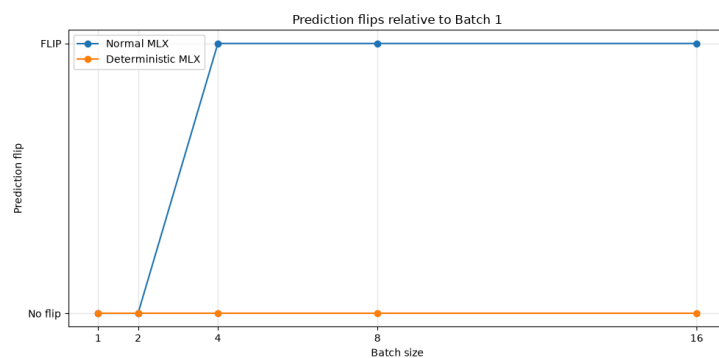


- Experiment 2 : Fixed prompt with patch sizes [1,2,4,8,16] , and 5 repeated run each , predictions flip from ‘Yes’ to ‘To’ at batch ≥ 4 , max Δ logits ≈ 0.094 , as shown :

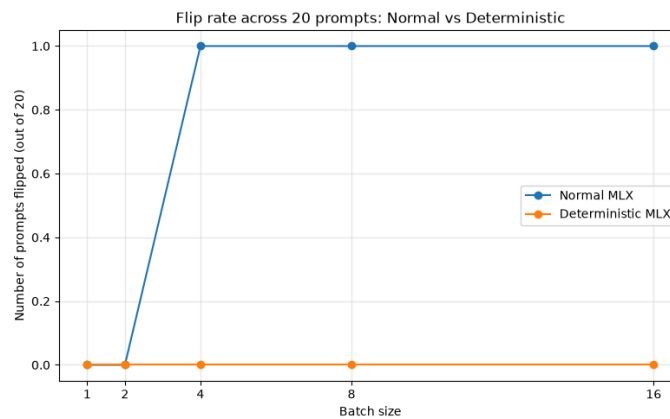


- **Experiment 3 : Deterministic (Batch Invariant) vs. normal mode , same prompt**
Repeating experiment 2, but this time with mlx-deterministic batch invariant kernel enabled. Result shows deterministic stabilised the decision on prompt.

Batch	Normal Δ	Det Δ	Normal Flip	Det Flip
1	0.0000000000e+00	0.0000000000e+00	False	False
2	6.2500000000e-02	1.2500000000e-01	False	False
4	9.3750000000e-02	8.2031250000e-02	True	False
8	7.0312500000e-02	8.2031250000e-02	True	False
16	7.8125000000e-02	7.4218750000e-02	True	False



Even with 20 prompts, the result shows “batch invariance helps stability of the final decision” (not reduce numerical noise)



Limitations

1. Singla small quantized model and limited set of single prompts.
2. Compute single token decision, not complete generation or safety/alignment behavior.
3. Deterministic stabilisation final decision, but not consistency reduces numerical logits.

Next Step

- Trying large set of prompts
- Replicated across models, batch sizes, seeds, numerical precisions
- Analyse the relationship between logit divergence, top-1/top-2 margin, decision flips.
- Test safety/refusal prompts and check whether it affects the safety-relevant behaviour.

Implementation Code

Experiment implementation are available in github :

<https://github.com/Shubbair/Batch-Invariant-Behaviour>

References:

[1] He, Horace and Thinking Machines Lab, "Defeating Nondeterminism in LLM Inference", *Thinking Machines Lab: Connectionism*, Sep 2025.

[2] https://github.com/thinking-machines-lab/batch_invariant_ops

[3] <https://github.com/ProbioticFarmer/mlx-deterministic>

[4] <https://huggingface.co/mlx-community/Qwen3-0.6B-4bit>